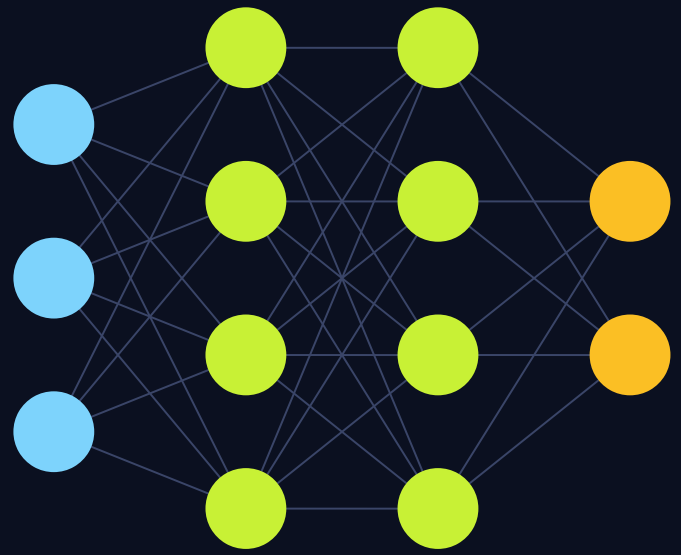
AIML 501 // VISUAL PRESENTATION

Inside a Neural Network

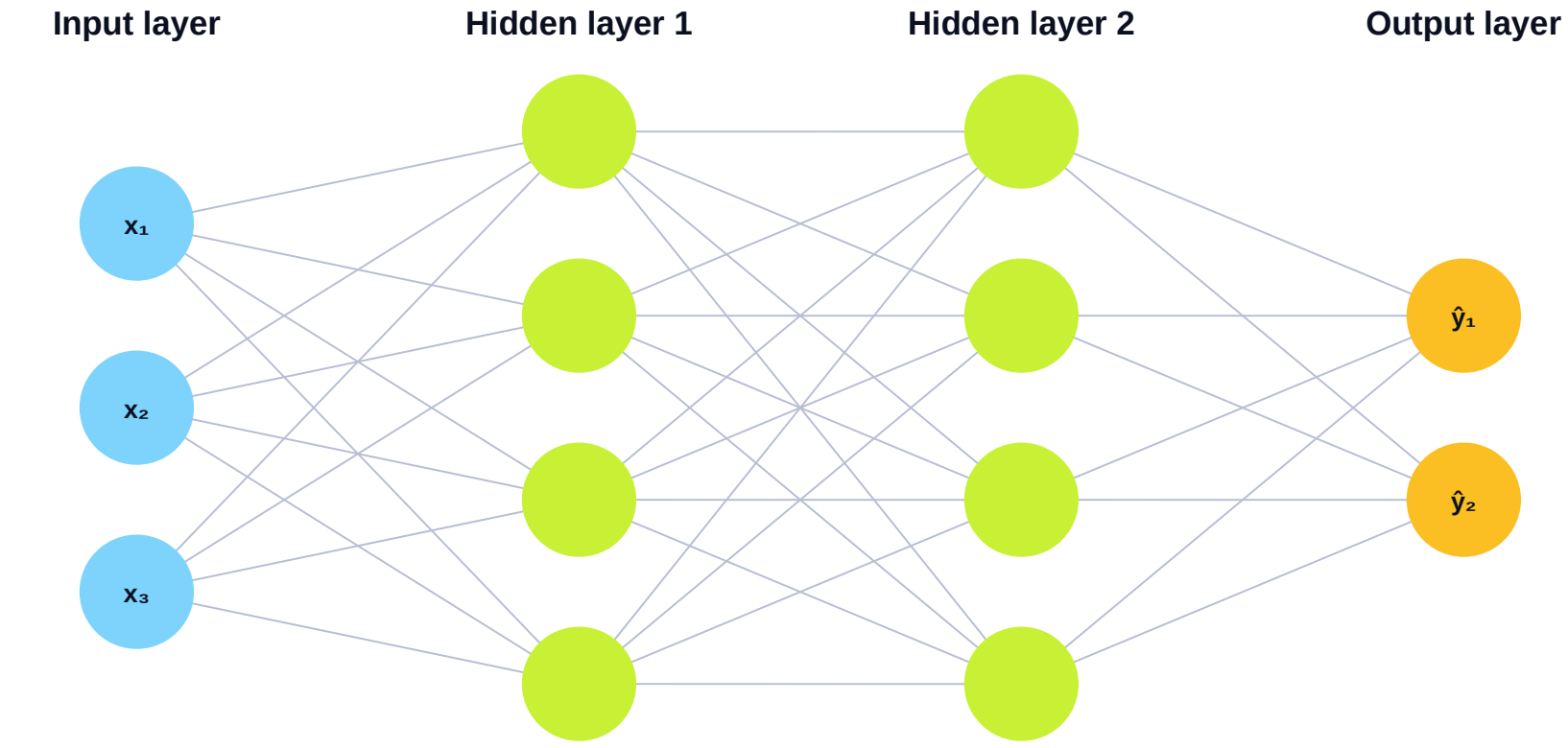


Six components, one flow: how data moves from input to prediction, and how the network learns from its mistakes.

Onwuosiuno Ikhioda

The Architecture: How Data Flows

A feedforward network passes data left to right. Every circle is a neuron; every line is a weighted connection.



Reading the diagram

- **Circles** are neurons: small calculators.
- **Columns** are layers: neurons that work in parallel.
- **Lines** are weights: how much one neuron influences the next.
- **Arrows** show direction. Training sends error back the other way.

Component 1: Layers

A layer is a group of neurons that work together on the same stage of processing.

1

Input layer

One neuron per feature. Holds the raw data and does no math.

Example: 784 pixels → 784 input neurons

2

Hidden layers

Where learning happens. Each layer refines what the previous one found. "Deep" learning means many hidden layers.

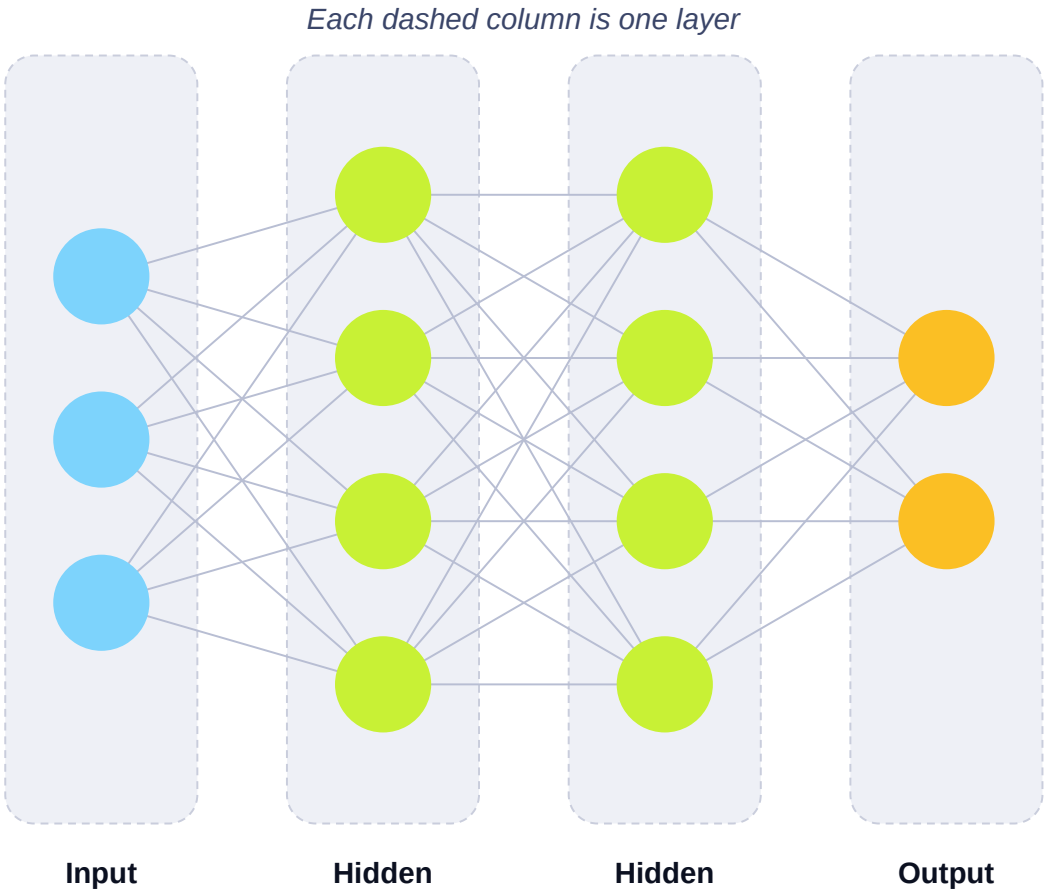
Example: Edges → shapes → faces

3

Output layer

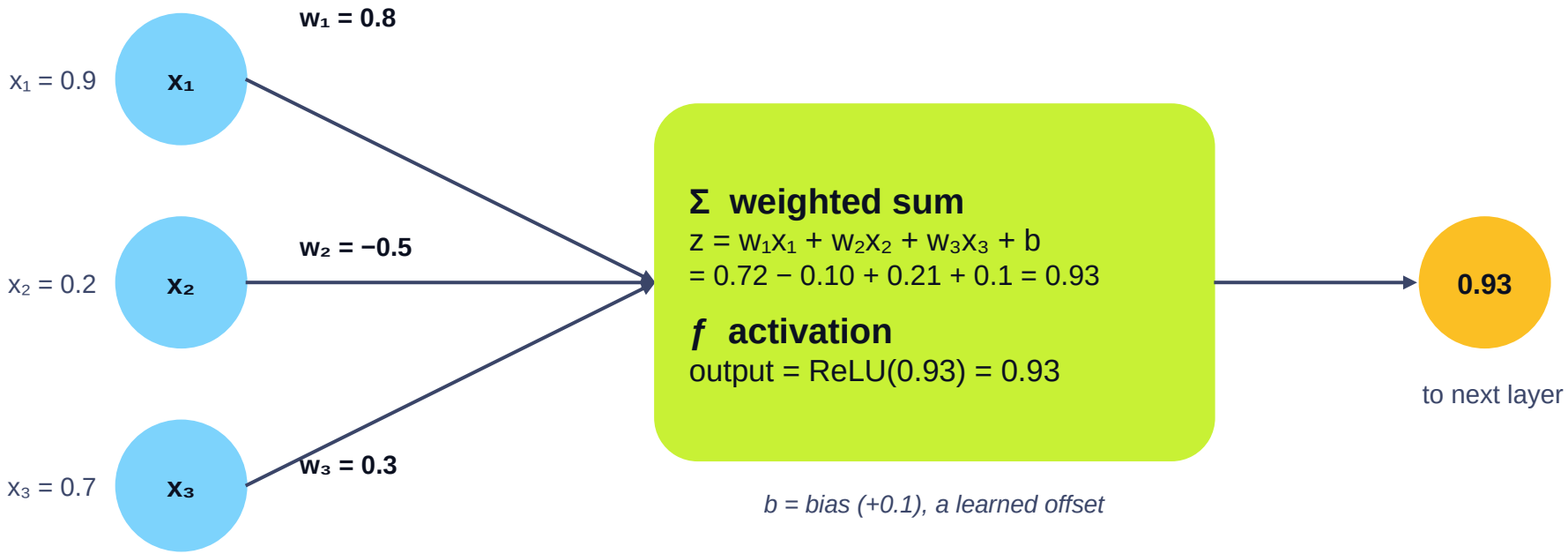
One neuron per answer. Produces the final prediction.

Example: 10 digits → 10 output neurons



Component 2: Neurons

A neuron is a unit that performs one basic calculation: weigh the inputs, add them up, decide what to pass on.



What it does

Multiplies each input by its weight, adds a bias, and passes the total through an activation function.

What it is not

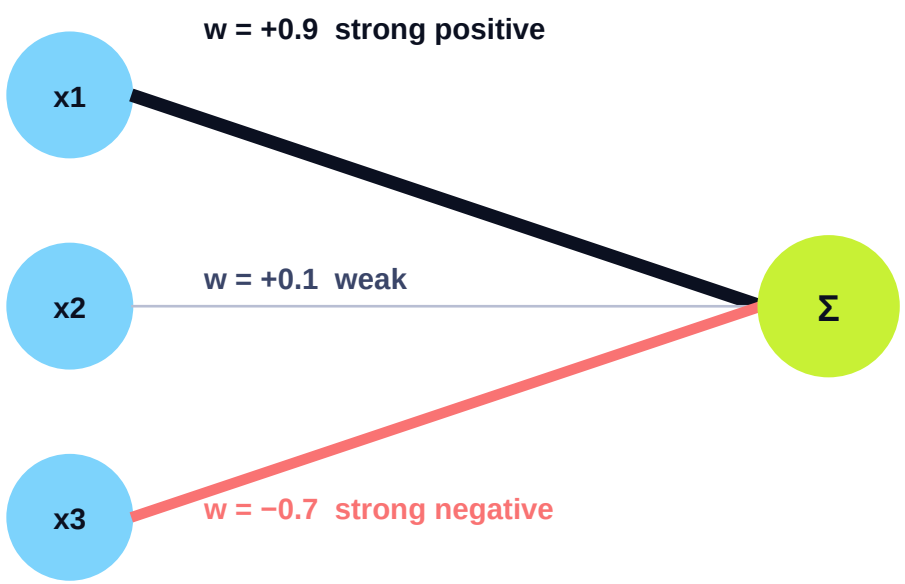
Not memory and not a rule. One neuron knows nothing on its own; meaning emerges from thousands working together.

Scale

A small image model has tens of thousands of neurons. Large language models have billions of weights between them.

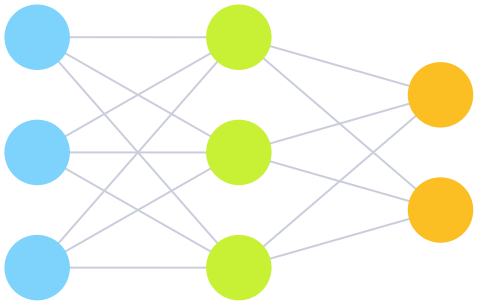
Component 3: Weights

Weights are the parameters that control how strongly each input influences a neuron. They are what the network learns.



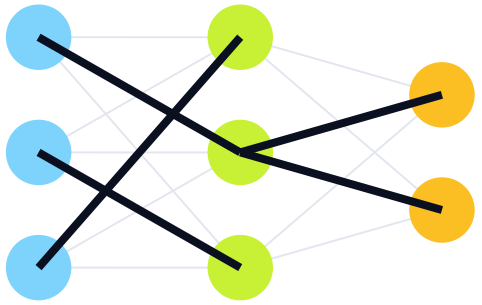
Line thickness = weight size. Red = negative (inhibits).

Before training



Random weights: every connection matters equally, predictions are noise.

After training



Learned weights: useful paths are strengthened, useless ones fade.

Why weights matter

The architecture is fixed; the weights are the knowledge. Training a network means finding the set of weights that makes the fewest mistakes.

Bias

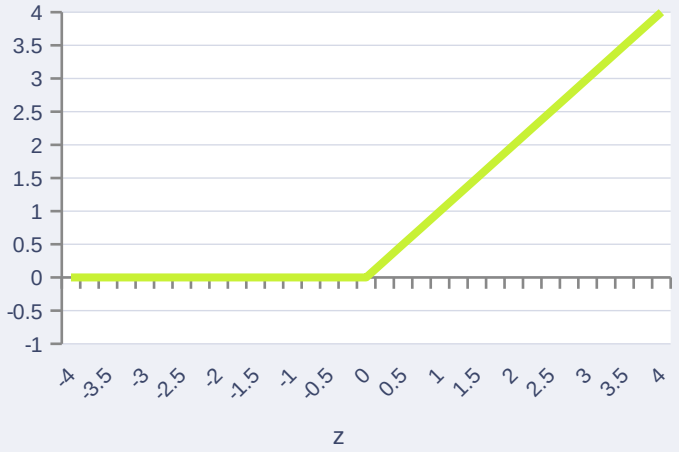
Each neuron also has a bias term: a weight on a constant input of 1 that shifts the activation threshold up or down.

Component 4: Activation Functions

An activation function decides what a neuron outputs given its weighted sum. It adds the non-linearity that lets networks learn curves, not just lines.

ReLU

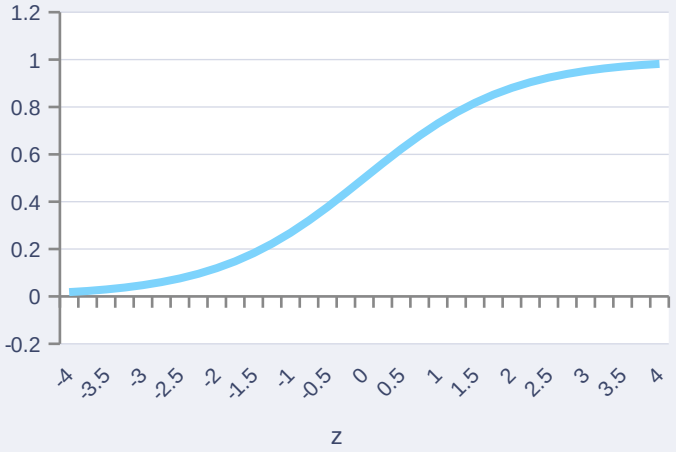
max(0, z)



Default for hidden layers. Fast, simple, avoids vanishing gradients.

Sigmoid

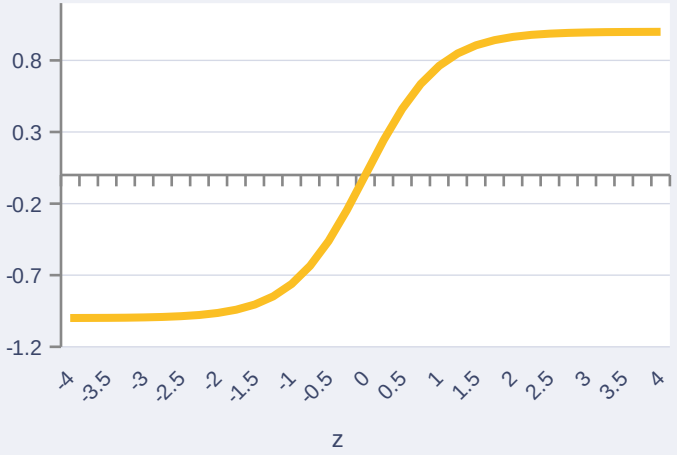
1 / (1 + e⁻ᶻ)



Squashes to 0–1. Used for yes/no outputs (probability).

Tanh

(eᶻ − e⁻ᶻ) / (eᶻ + e⁻ᶻ)



Squashes to −1 to 1. Zero-centered; common in recurrent nets.

Component 5: Loss Functions

A loss function measures how wrong the prediction is. It turns "accuracy" into a single number the network can try to shrink.

Example: classifying one image as cat, dog, or bird

	cat	dog	bird
Prediction ŷ	**0.70**	**0.20**	**0.10**
Truth y	**1.00**	**0.00**	**0.00**

Cross-entropy loss = −log(0.70) = 0.36
If the model had predicted 0.95 for cat, loss would drop to 0.05. If 0.10, loss would jump to 2.30.

Mean Squared Error (regression)

Average of (prediction − truth)². Penalizes big misses heavily. Used when predicting a number such as a price or temperature.

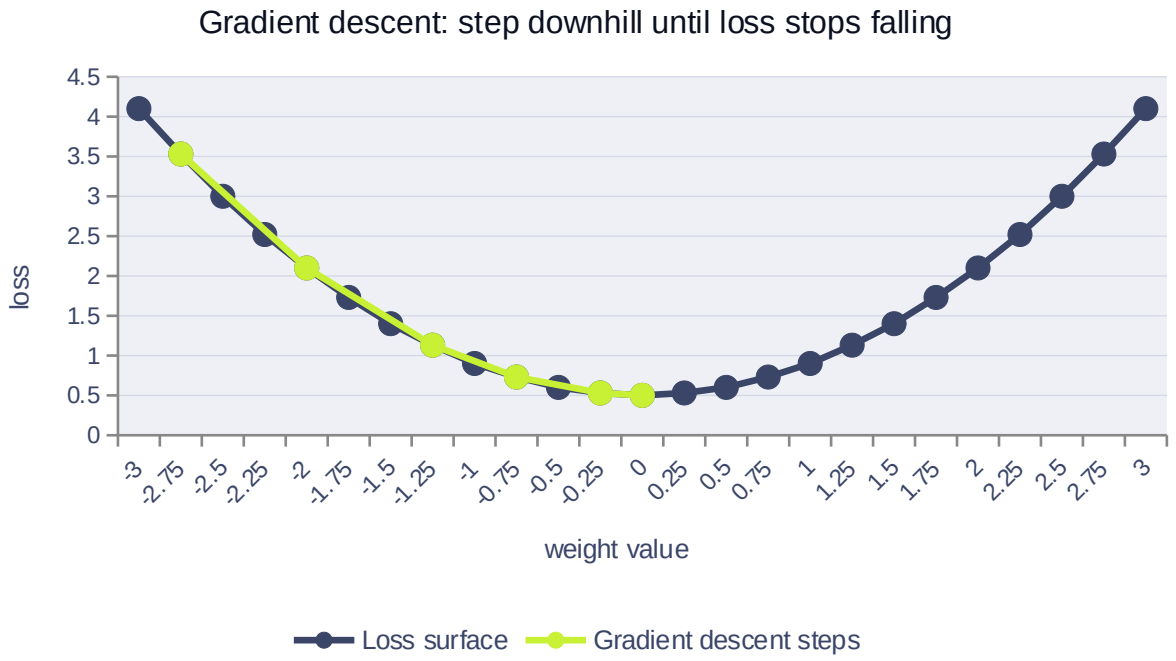
Cross-Entropy (classification)

Measures how far the predicted probabilities are from the true class. Confident wrong answers cost the most. Used for labels such as spam / not spam.

Key idea: the loss function is the network's only feedback. It never sees "right" or "wrong," only a number to make smaller. Choosing the wrong loss means optimizing the wrong goal.

Component 6: Optimization Algorithms

An optimizer adjusts the weights to reduce the loss. It is the mechanism that turns error into learning.



w ← w − η · ∂L/∂w

new weight = old weight − learning rate × slope of the loss. Backpropagation computes the slope for every weight at once.

1

SGD

Stochastic Gradient Descent. One small batch at a time; simple and reliable.

2

Momentum

Remembers the previous direction so it rolls through small bumps instead of stalling.

3

Adam

Adapts the learning rate per weight. The default choice for most deep learning today.

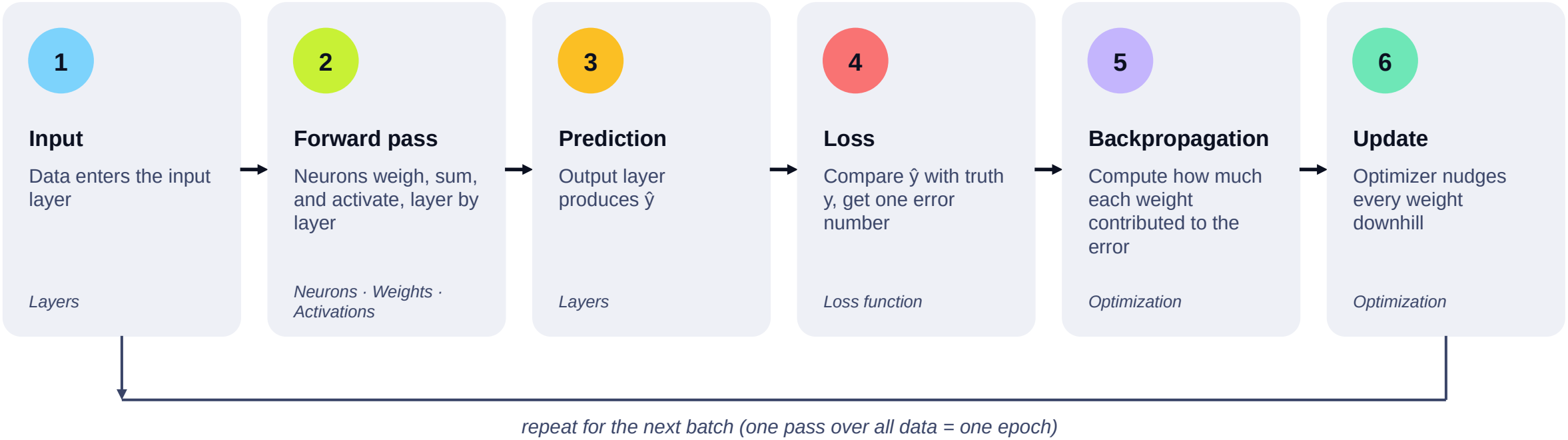
4

Learning rate η

Step size. Too big overshoots the minimum; too small takes forever to arrive.

How the Six Components Work Together

One training step. Repeat it thousands of times and the weights converge on a useful model.



Forward pass uses layers, neurons, weights, and activations to make a guess. **Backward pass** uses the loss function and optimizer to improve the guess.

Summary: Why Visualize Neural Networks

It makes the abstract concrete

A weight is a line, a neuron is a circle, a layer is a column. Once you can point at it, you can reason about it.

It shows where learning lives

The architecture never changes during training. Drawing it makes clear that only the weights move.

It separates the two passes

Seeing data flow left and error flow right explains why forward and backward propagation are different jobs.

Key insights from this exercise

- A neural network is six simple ideas stacked, not one complicated one.
- Activation functions are what make depth worth having; without them every layer collapses into one.
- The loss function defines the goal. Pick the wrong one and the optimizer will faithfully learn the wrong thing.
- Backpropagation is just the chain rule applied in reverse along the same lines the data came in on.
- Explaining each piece in one sentence was harder than using them in code, and more valuable.